

Reporte Práctica Análisis del Algoritmo Stable Matching Problem

Metztli Donaji Pablo Aparicio
Algorithm, Center for Computer Investigation
Instituto Politécnico Nacional
mpabloa26@cic.mx

Abstract—El análisis del algoritmo del Stable Matching desde el punto de vista de Gale-Shapley es un fenómeno interesante de observar en cuanto al comportamiento en cuestión del tiempo se trata, debido a que se requiere revisar una gran suma de datos para poder ver el comportamiento del algoritmo conforme pasa el tiempo. Actualmente con la tecnología es posible visualizar este comportamiento gracias a la programación con la cual podemos realizar este análisis en relativamente poco tiempo, que es lo que se estará revisando en este reporte.

Index Terms—Algoritmo de Gale-Shapley, complejidad temporal, análisis, simulación.

I. INTRODUCCIÓN

El problema del Stable Matching, también conocido como problema del matrimonio estable, fue introducido formalmente por David Gale y Lloyd Shapley en 1962 [1]. Este problema busca emparejar dos conjuntos de igual tamaño de manera que no exista una *pareja bloqueante*, es decir, dos elementos que prefieran estar juntos antes que con sus parejas asignadas. Su solución garantiza que todos los elementos queden emparejados con una alta tasa de satisfacción respecto a sus listas de preferencias dadas.

La relevancia práctica de este algoritmo es amplia: se aplica en la asignación de residentes médicos a hospitales, en sistemas de admisión universitaria, en mercados de trabajo y en protocolos de intercambio de redes [2]. De hecho, Lloyd Shapley recibió el Premio Nobel de Economía en 2012 junto con Alvin Roth, en parte por estas contribuciones al estudio de los mercados con emparejamiento estable [3].

El algoritmo propuesto por Gale y Shapley garantiza la obtención de un matching estable para cualquier instancia del problema, con una complejidad temporal en el peor de los casos de $O(n^2)$ propuestas [1]. El procedimiento consiste en que los proponentes realizan ofertas sucesivas siguiendo su lista de preferencias, mientras que los receptores aceptan provisionalmente la mejor oferta recibida hasta ese momento y rechazan las demás. Este proceso se repite hasta que todos los elementos han sido emparejados [4].

Sin embargo, la notación asintótica $O(n^2)$ describe el comportamiento del algoritmo a medida que el tamaño del problema tiende a infinito, pero no detalla su rendimiento en la práctica para tamaños de entrada finitos. Estudios empíricos han mostrado que el número promedio de propuestas tiende a ser significativamente menor que el peor caso, aproximándose a $n \ln n$ cuando las preferencias son aleatorias [4]. Por ello,

en este reporte se realiza un análisis empírico del algoritmo midiendo los tiempos reales de ejecución para distintos valores de n , con el objetivo de contrastar el comportamiento teórico con el práctico y comprender mejor el impacto de cada etapa del proceso.

II. OBJETIVOS

El objetivo general de esta práctica es analizar empíricamente el comportamiento temporal del algoritmo de Gale-Shapley en función del tamaño de la entrada, contrastando los resultados obtenidos con la complejidad teórica $O(n^2)$.

Objetivos específicos son:

- Implementar y validar el algoritmo de Gale-Shapley en Python para instancias de distinto tamaño.
- Generar conjuntos de datos de prueba con listas de preferencias aleatorias para n desde 0 hasta 8100.
- Medir y comparar los tiempos de ejecución en diferentes etapas del proceso, enfocados a: generación de archivos, lectura de datos y ejecución del algoritmo.
- Visualizar el crecimiento de los tiempos para identificar el tipo de crecimiento de cada etapa.

III. MARCO TEÓRICO

A. Definiciones Formales

Sea P un conjunto de n proponentes y R un conjunto de n receptores. Cada elemento $p \in P$ tiene una lista de preferencias estricta sobre todos los elementos de R , y cada $r \in R$ tiene una lista de preferencias estricta sobre todos los elementos de P .

Un *matching* M es una función inyectiva $M : P \rightarrow R$ que empareja a cada proponente con exactamente un receptor. Se dice que un par $(p, r) \notin M$ es una *pareja bloqueante* si:

- p prefiere a r sobre su pareja actual $M(p)$, y
- r prefiere a p sobre su pareja actual $M^{-1}(r)$.

Un matching es **estable** si no existe ninguna pareja bloqueante [1].

B. Algoritmo de Gale-Shapley

El algoritmo de Gale-Shapley, también llamado *deferred acceptance* (aceptación diferida), opera de la siguiente manera: en cada ronda, cada proponente libre realiza una propuesta al siguiente receptor en su lista de preferencias que aún no lo ha rechazado. Cada receptor que recibe propuestas retiene

provisionalmente la mejor según sus preferencias y rechaza al resto. Los proponentes rechazados pasan a la siguiente ronda como libres. El proceso termina cuando no quedan proponentes libres [4].

1) *Correctez*: El algoritmo termina siempre en a lo más n^2 rondas, ya que cada proponente realiza a lo sumo n propuestas y nunca repite una propuesta a un receptor que ya lo rechazó. El matching resultante es siempre estable: si existiera una pareja bloqueante (p, r) , entonces p habría propuesto a r antes que a su pareja actual, y r lo habría aceptado o retenido, lo cual contradice la suposición [1].

2) *Complejidad*: La complejidad temporal del algoritmo en el peor caso es $O(n^2)$, pues el número total de propuestas no puede exceder n^2 . La complejidad espacial es $O(n^2)$ para almacenar las listas de preferencias y las tablas de rango [4].

IV. ENTORNO DE PRUEBAS

Las mediciones de tiempo reportadas en este documento fueron obtenidas bajo el siguiente entorno de hardware y software:

- **Sistema operativo:** macOS y windows 11
- **Lenguaje:** Python 3.11
- **Medición de tiempo:** módulo time de la biblioteca estándar (`time.time()`)
- **Almacenamiento:** archivos de texto plano, uno por instancia

V. PROCEDIMIENTO Y DESARROLLO

Se desarrolló primero el código para el algoritmo de Gale-Shapley en el cual se realizaron pruebas con datos pequeños de 3 hombres y 3 mujeres, de tal forma que funcionara escogiendo el género con el que empieza a emparejar y el número de hombres y mujeres que estarían participando. Una vez que se comprobó que el código funcionara de manera correcta, se realizó la siguiente fase: generar los archivos de entrada a través de código, dando como parámetro el número de archivos a crear. Cada archivo contiene la lista de hombres y mujeres con su respectiva lista de preferencias.

Proseguimos con la fase de automatización para la lectura de estos archivos con datos generados aleatoriamente. Al final se integró todo en un archivo `main.py` que también devuelve una tabla en terminal con el número de archivos, el número de parejas n , y los tiempos medidos en cada etapa. Los datos obtenidos se muestran en la Tabla I.

TABLE I
TIEMPOS DE GENERACIÓN, LECTURA Y EJECUCIÓN DEL ALGORITMO (SEGUNDOS)

Archivo	n	Generación (s)	Lectura (s)	Algoritmo (s)
1	0	9.829529	0.000153	0.000006
2	300	9.829529	0.018509	0.007038
3	600	9.829529	0.074924	0.028268
4	900	9.829529	0.137381	0.072499
5	1200	9.829529	0.242046	0.125089
6	1500	9.829529	0.351641	0.197649
7	1800	9.829529	0.490303	0.266844
8	2100	9.829529	0.726004	0.353211
9	2400	9.829529	0.963746	0.460825
10	2700	9.829529	1.437186	0.597537
11	3000	9.829529	1.622117	0.838367
12	3300	9.829529	1.944287	0.941092
13	3600	9.829529	2.200223	1.079213
14	3900	9.829529	3.092964	1.402347
15	4200	9.829529	3.334435	1.534270
16	4500	9.829529	4.201768	1.995398
17	4800	9.829529	6.357785	2.292182
18	5100	9.829529	5.824439	2.567064
19	5400	9.829529	8.420512	2.721346
20	5700	9.829529	10.259144	3.826991
21	6000	9.829529	9.080085	3.471886
22	6300	9.829529	9.144080	4.125159
23	6600	9.829529	10.888870	4.523089
24	6900	9.829529	14.321148	4.973410
25	7200	9.829529	16.719456	6.196266
26	7500	9.829529	20.774941	6.788633
27	7800	9.829529	25.169671	6.860938
28	8100	9.829529	51.380762	7.304061

La forma en que se pedían y daban los datos a través de la terminal fue de la siguiente manera:

género que empieza? m
número de archivos: 28

Las gráficas resultantes se muestran en las Figuras 1, 2 y 3.

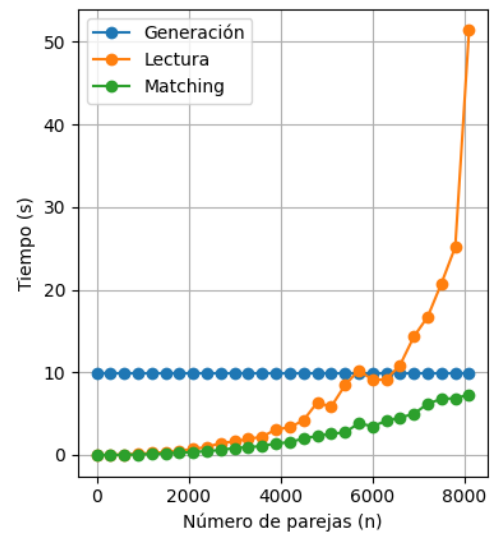


Fig. 1. Escala lineal: número de parejas n vs tiempo (s). Generación (azul), lectura (naranja) y algoritmo de Gale-Shapley (verde).

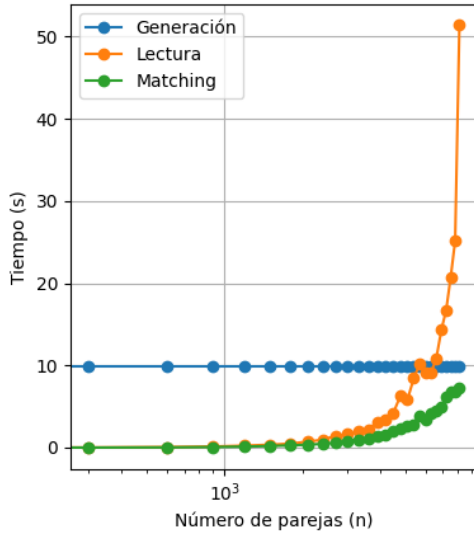


Fig. 2. Escala lineal en el eje x : número de parejas n vs tiempo (s). Generación (azul), lectura (naranja) y algoritmo de Gale-Shapley (verde).

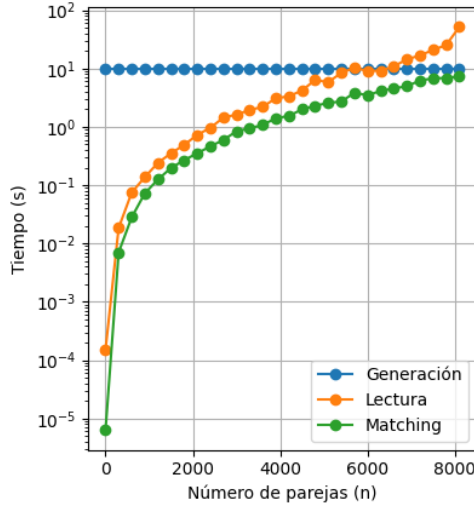


Fig. 3. Escala logarítmica en el eje y : número de parejas n vs tiempo (s). Generación (azul), lectura (naranja) y algoritmo de Gale-Shapley (verde).

Algoritmo Gale-Shapley

```

0: procedure GALESHPLEY( $n, Pref_P, Rank_R$ )
0:   Inicializar arreglos de emparejamiento
0:   Conjunto  $F$  de proponentes libres
0:   while  $F \neq \emptyset$  do
0:     Seleccionar  $p \in F$ 
0:      $r \leftarrow$  siguiente preferencia de  $p$ 
0:     if  $r$  está libre then
0:       Emparejar  $p$  con  $r$ 
0:     else
0:       Comparar rangos y actualizar emparejamiento si
es necesario
0:     end if
0:   end while
0:   Retornar emparejamiento

```

0: **end procedure**=0

VI. ANÁLISIS DE RESULTADOS

A. Etapa de Generación

Como se observa en la Figura 1, el tiempo de generación de archivos permanece prácticamente constante en todos los casos (≈ 9.83 s). Esto se debe a que el proceso de escritura en disco depende principalmente del hardware de almacenamiento y no del tamaño de la instancia individual, comportamiento consistente con una complejidad $O(1)$ respecto a n .

B. Etapa de Lectura

La lectura de archivos presenta el crecimiento más pronunciado de las tres etapas. En la Figura 1, se observa que la curva de lectura crece de forma superlineal con respecto a n . Esto se explica porque al aumentar n , no solo aumenta el tamaño de cada archivo, sino también el tiempo de parseo de las listas de preferencias en memoria. El valor atípico en $n = 8100$ (archivo 28, ≈ 51.38 s) sugiere la presencia de presión de memoria o contención de I/O en esa ejecución particular, posiblemente por interferencia del sistema operativo.

Al observar la Figura 2 con escala logarítmica en el eje x , la curva de lectura se aproxima a una línea recta, indicando un crecimiento de tipo polinomial consistente con $O(n^2)$, dado que el tamaño de cada archivo crece con n^2 .

C. Etapa del Algoritmo

El tiempo de ejecución del algoritmo de Gale-Shapley también crece con n , pero de manera más suave que la lectura, como se aprecia en la Figura 1. En la Figura 3, al aplicar escala logarítmica en el eje y , ambas curvas muestran una pendiente similar, confirmando que ambas tienen un orden de crecimiento comparable. Sin embargo, los valores absolutos del algoritmo son consistentemente menores que los de la lectura, lo que indica que el cuello de botella principal del proceso es la etapa de entrada/salida y no el cómputo del emparejamiento.

Este resultado es relevante desde una perspectiva de optimización: para reducir los tiempos totales, sería más efectivo mejorar la eficiencia de la lectura (por ejemplo, usando formatos binarios o estructuras de datos en memoria compartida) que optimizar el propio algoritmo de emparejamiento.

VII. CONCLUSIÓN

En esta práctica se implementó, validó y analizó empíricamente el algoritmo de Gale-Shapley para el problema del Stable Matching. La validación inicial con instancias pequeñas ($n = 3$) confirmó la correctez de la implementación, y las pruebas a gran escala (hasta $n = 8100$) permitieron observar el comportamiento temporal real del algoritmo.

Los resultados muestran que la generación de archivos tiene un tiempo casi constante independientemente de n , mientras que tanto la lectura como la ejecución del algoritmo presentan un crecimiento de tipo polinomial, consistente con la complejidad teórica $O(n^2)$. El hallazgo más significativo es que el tiempo de lectura supera al tiempo de ejecución del

algoritmo en todos los casos medidos, lo que indica que el cuello de botella está en la etapa de entrada/salida y no en el cómputo del emparejamiento.

Esto sugiere que en aplicaciones prácticas con grandes volúmenes de datos, una optimización prioritaria sería el diseño de estructuras de almacenamiento más eficientes, como formatos binarios compactos o la generación de preferencias directamente en memoria, evitando la escritura y lectura de archivos de texto. El comportamiento del algoritmo en sí es robusto y su crecimiento empírico se mantiene por debajo del peor caso teórico $O(n^2)$, en línea con los resultados reportados en la literatura [4].

REFERENCES

- [1] D. Gale and L. S. Shapley, "College Admissions and the Stability of Marriage," *The American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, Jan. 1962.
- [2] A. E. Roth and M. A. O. Sotomayor, *Two-Sided Matching: A Study in Game-Theoretic Modeling and Analysis*. Cambridge University Press, 1990.
- [3] The Royal Swedish Academy of Sciences, "The Prize in Economic Sciences 2012," Nobel Prize, Oct. 2012. [Online]. Available: <https://www.nobelprize.org/prizes/economic-sciences/2012/summary/>. [Accessed: Oct. 31, 2025].
- [4] D. E. Knuth, *Stable Marriage and Its Relation to Other Combinatorial Problems*. American Mathematical Society, 1997.